16 EXCEPTION HANDLING • DETAILED SOLUTIONS

output solutions

Detailed Theory, Exact C++ Rules, Memory Models & Exam Reference

Detailed Solutions — Predict the Output & Debugging (Module 16: Exception Handling)

Solution to Q5

- `item a(1)` prints `C1 `.
- `item b(2)` prints `C2 `.
- `throw 10;` triggers stack unwinding before `item c(3)` is reached: `b` is destroyed first (`D2 `), then `a` is destroyed (`D1 `).
- `catch (int x)` prints `Caught:10 `, followed by `End `.
- **Guaranteed Output**:

```text
C1 C2 D2 D1 Caught:10 End
```

Solution to Q6

- Because `e` is caught by `const` reference (`const base_err& e`) and `msg()` is `virtual`, dynamic dispatch calls `derived_err::msg()`.
- **Guaranteed Output**:

```text
DerivedError
```

Solution to Q7 (Debugging Handler Ordering)

- **Bug**: `std::out_of_range` inherits from `std::exception`. Because `catch (const exception& e)` appears **first**, it catches every `out_of_range` exception and makes the second `catch (const out_of_range& e)` unreachable dead code!
- **Fix**: Place `catch (const out_of_range& e)` **before** `catch (const exception& e)`.

Solution to Q8 (Debugging Memory Leak on `throw`)

- **Bug**: When `x < 0`, `throw "Negative input";` exits `process()` before `delete[] buffer;` is reached, leaking 50 `int`s on the heap.
- **Fix**: Use `std::unique_ptr<int[]> buffer = std::make_unique<int[]>(50);` (or `std::vector<int> buffer(50);`) so stack unwinding frees the memory automatically.